

Documentação Técnica

Sistema de Gestão de Armazém

.NET • Dapper • SQL Server • React • TypeScript • Tailwind CSS

Versão 2.0 — Agosto de 2026

Atualizada para refletir o sistema efetivamente entregue (a Versão 1.0 era o planejamento inicial)

Sumário

1. Visão Geral do Projeto	3
2. Requisitos Funcionais	3
3. Requisitos Não Funcionais	4
4. Stack Tecnológica	4
5. Arquitetura	5
6. Glossário	6
7. Atores e Perfis de Acesso	6
8. Casos de Uso	7
9. Regras de Negócio	8
10. Modelo de Dados	8
11. Contratos de API	10
12. Plano de Testes (critérios de aceite)	11
13. Riscos e Mitigações	12
14. Status do Projeto	12
15. Próximos Passos	13

1. Visão Geral do Projeto

O sistema de Gestão de Armazém controla o fluxo de produtos dentro de um ou mais armazéns, desde o recebimento de mercadorias até a expedição, passando pelo controle de estoque, localização física dos itens, ajustes de inventário e rastreabilidade completa das movimentações.

O sistema é composto por uma API back-end em .NET, persistência em SQL Server via Dapper, e uma interface web em React + TypeScript com Tailwind CSS. Esta versão do documento descreve o sistema como ele foi efetivamente entregue — não mais um planejamento inicial.

1.1 Objetivos

- Controlar entrada, saída, transferência e ajuste de produtos entre localizações.
- Manter saldo de estoque em tempo real por produto e por localização, em múltiplos armazéns.
- Registrar histórico auditável e imutável de todas as movimentações, incluindo o motivo de ajustes de inventário.
- Gerenciar pedidos de recebimento (fornecedores) e expedição (clientes), incluindo cancelamento enquanto em aberto.
- Oferecer controle de acesso por perfil de usuário, com gestão de usuários e redefinição de senha.
- Manter sessões renovadas automaticamente (refresh token), sem exigir login repetido.

1.2 Escopo entregue

Escopo completo: cadastro de produtos, categorias, armazéns e localizações (com edição e exclusão); controle de estoque; movimentações de entrada, saída, transferência e ajuste; pedidos de recebimento e expedição com cancelamento; autenticação com renovação automática de sessão; permissões por perfil de usuário; múltiplos armazéns; relatórios de estoque baixo e histórico de movimentações. Diferente da Versão 1.0 deste documento — que projetava múltiplos armazéns e relatórios avançados para uma fase posterior — ambos foram entregues nesta versão do sistema.

Deploy em ambiente de produção está fora do escopo desta entrega (ver Seção 15).

2. Requisitos Funcionais

Código	Descrição	Prioridade
RF01	Cadastrar, editar, excluir e consultar produtos (SKU, código de barras, categoria, unidade de medida, estoque mínimo)	Alta
RF02	Cadastrar, editar e excluir categorias de produto	Média
RF03	Cadastrar, editar e excluir armazéns e localizações (corredor, prateleira, nível)	Alta
RF04	Registrar entrada de produtos (recebimento)	Alta
RF05	Registrar saída de produtos (expedição)	Alta
RF06	Registrar transferência de produtos entre localizações	Média
RF07	Registrar ajuste de estoque a partir de uma contagem física de inventário, com motivo opcional	Média

Código	Descrição	Prioridade
RF08	Consultar saldo de estoque por produto e por localização	Alta
RF09	Gerenciar pedidos de recebimento: criar, confirmar recebimento item a item, cancelar enquanto em aberto	Média
RF10	Gerenciar pedidos de expedição: criar, expedir (tudo ou nada), cancelar enquanto em aberto	Média
RF11	Manter histórico imutável de movimentações de estoque (auditoria)	Alta
RF12	Autenticar usuários via JWT, com renovação automática de sessão (refresh token) e limite de tentativas de login	Alta
RF13	Gerenciar usuários e perfis de acesso: cadastrar, editar, excluir, redefinir senha (Administrador); trocar a própria senha (qualquer usuário)	Alta
RF14	Emitir relatórios de estoque abaixo do mínimo e de histórico de movimentações com filtros	Média

3. Requisitos Não Funcionais

Categoria	Requisito
Desempenho	Consultas de saldo de estoque devem responder em até 500ms sob carga normal. Listagens paginadas (Produtos, Pedidos) evitam carregar o conjunto inteiro de uma vez.
Segurança	Autenticação via JWT com refresh token e rotação; senhas com hash bcrypt; HTTPS obrigatório; rate limiting no login (5 tentativas/minuto/IP); a API recusa subir fora de Development com a chave JWT padrão.
Concorrência	Baixas simultâneas no mesmo item de estoque não podem gerar saldo negativo indevido (controle otimista via RowVersion).
Auditoria	Movimentações de estoque não podem ser alteradas ou excluídas, apenas inseridas — incluindo ajustes de inventário, que registram o motivo.
Observabilidade	Logging estruturado (Serilog) de todas as requisições; health check de conectividade com o banco (GET /health).
Usabilidade	Interface responsiva; confirmações e mensagens de erro/sucesso via diálogos e toasts in-app, não pop-ups nativos do navegador.
Escalabilidade	Modelo de dados suporta múltiplos armazéns sem redesenho de schema (entregue).
Testabilidade	Regras de negócio cobertas por testes unitários (mock dos repositórios) e testes de integração (banco real), além de testes de front-end para toda página e componente.

4. Stack Tecnológica

Camada	Tecnologia
Back-end	.NET 8 (Web API)
Acesso a dados	Dapper (Micro-ORM)

Camada	Tecnologia
Banco de dados	SQL Server
Versionamento de schema	DbUp
Autenticação	JWT + refresh token com rotação (bcrypt para hash de senha)
Observabilidade	Serilog (logs estruturados: console + arquivo), health check
Segurança	Rate limiting no login (Microsoft.AspNetCore.RateLimiting)
Front-end	React 19 + TypeScript, Vite 8
Estilização	Tailwind CSS v4
Testes back-end	xUnit + Moq (unitários); testes de integração contra SQL Server real
Testes front-end	Vitest + Testing Library

5. Arquitetura

O back-end segue os princípios da Clean Architecture, organizado em camadas com responsabilidades e dependências bem definidas, permitindo testar regras de negócio isoladamente da infraestrutura.

Camada	Responsabilidade
Domain	Entidades e interfaces de repositório; não depende de nenhuma outra camada.
Application	Casos de uso, DTOs, validators (FluentValidation) e serviços; orquestra o Domain.
Infrastructure	Implementação dos repositórios com Dapper, segurança (bcrypt, JWT) e conexão com o SQL Server.
API	Controllers, middleware de exceções, autenticação, rate limiting, health check, injeção de dependência.
Frontend	SPA em React/TypeScript que consome a API via REST, com renovação automática de sessão.

Fluxo de dependência: Frontend → API → Application → Domain. A Infrastructure implementa as interfaces definidas no Domain e é injetada na Application em tempo de execução.

5.1 Estrutura de pastas — Back-end

- GestaoArmazem.sln
- src/GestaoArmazem.Domain — entidades, enums, interfaces de repositório
- src/GestaoArmazem.Application — DTOs, services, validators
- src/GestaoArmazem.Infrastructure — repositórios Dapper, segurança
- src/GestaoArmazem.API — controllers, Program.cs, health check, appsettings
- src/GestaoArmazem.Database — CLI de migração (DbUp), aplica src/database/scripts
- src/database/scripts — scripts SQL versionados (fonte única de verdade do schema)
- tests/GestaoArmazem.Application.Tests — testes unitários (repositórios mockados)

- tests/GestaoArmazem.IntegrationTests — testes de integração (SQL Server real, sem Docker)

5.2 Estrutura de pastas — Front-end

- src/api — client HTTP (com renovação automática de token) + chamadas por recurso
- src/components/layout — Sidebar, AppShell, ProtectedRoute, AdminRoute
- src/components/ui — Button, Input, Select, Alert, StatusBadge, Pagination
- src/context — AuthContext, ToastContext, DialogContext
- src/lib — utilitários (decodificação de JWT, formatação de localização)
- src/pages — uma página por rota (21 páginas)
- src/types — tipos espelhando os DTOs do back-end
- App.tsx — definição das rotas, incluindo rotas restritas por perfil

6. Glossário

Termo	Definição
WMS	Warehouse Management System — sistema de gestão de armazém.
SKU	Stock Keeping Unit — código único que identifica um produto.
Localização (bin)	Endereço físico dentro do armazém (corredor, prateleira, nível) onde um produto é guardado.
Movimentação	Registro de qualquer entrada, saída, transferência ou ajuste de estoque.
Ajuste de estoque	Movimentação que corrige o saldo para o valor exatamente contado numa conferência física de inventário.
Saldo de estoque	Quantidade disponível de um produto em uma localização específica.
Pedido de recebimento	Solicitação de compra a um fornecedor, aguardando chegada de mercadoria.
Pedido de expedição	Solicitação de saída de mercadoria destinada a um cliente.
Refresh token	Token de longa duração usado para renovar a sessão (access token JWT) sem exigir novo login.
Rate limiting	Limite do número de requisições aceitas num intervalo de tempo — usado no login para mitigar força bruta de senha.

7. Atores e Perfis de Acesso

Ator	Descrição
Operador de Armazém	Registra entradas, saídas, transferências e ajustes no dia a dia da operação.
Gestor de Estoque	Consulta saldos e relatórios, gerencia pedidos de recebimento e expedição, incluindo cancelamento.
Administrador	Gerencia usuários, perfis e permissões — único ator autorizado a essas ações (RN07).

Permissão é validada pelo nome do perfil do usuário autenticado (claim "role" do JWT), não por uma tabela de permissões granulares — ver observação na Seção 10.

8. Casos de Uso

Código	Nome	Ator principal	Resumo
UC01	Registrar entrada de produto	Operador de Armazém	Dá entrada em produtos recebidos, associando-os a uma localização.
UC02	Registrar saída de produto	Operador de Armazém	Baixa produtos do estoque.
UC03	Transferir produto entre localizações	Operador de Armazém	Move produtos entre localizações, inclusive de armazéns diferentes.
UC04	Consultar saldo de estoque	Gestor de Estoque	Consulta quantidade disponível por produto e localização.
UC05	Gerenciar pedido de recebimento	Gestor de Estoque	Cria, acompanha, confirma itens e cancela pedidos de compra.
UC06	Gerenciar pedido de expedição	Gestor de Estoque	Cria, expede (tudo ou nada) e cancela pedidos de saída.
UC07	Gerenciar usuários e perfis	Administrador	Cadastra, edita, exclui usuários; redefine senhas; gerencia perfis.
UC08	Ajustar estoque	Operador de Armazém / Gestor de Estoque	Corrige o saldo de um produto numa localização para o valor contado fisicamente.
UC09	Cancelar pedido	Gestor de Estoque	Cancela um pedido de recebimento ou expedição, somente enquanto em aberto (RN09).

8.1 UC01 — Registrar entrada de produto (fluxo detalhado)

Pré-condição: produto e localização já cadastrados.

- 1. Operador informa o produto e a quantidade recebida.
- 2. Sistema valida se o produto e a localização existem.
- 3. Operador confirma a localização de destino.
- 4. Sistema cria um registro em `MovimentacaoEstoque` do tipo `Entrada`.
- 5. Sistema atualiza (ou cria) o saldo em Estoque para o par produto/localização.

Pós-condição: saldo de estoque atualizado e movimentação registrada de forma auditável.

8.2 UC03 — Transferir produto entre localizações (fluxo detalhado)

Pré-condição: saldo suficiente do produto na localização de origem.

- 1. Operador informa produto, localização de origem, localização de destino e quantidade.
- 2. Sistema valida se há saldo suficiente na origem (RN01).
- 3. Sistema abre uma transação de banco de dados.
- 4. Sistema debita o saldo da localização de origem.

- 5. Sistema credita o saldo da localização de destino.
- 6. Sistema registra uma única MovimentacaoEstoque do tipo Transferência, com origem e destino preenchidos.
- 7. Sistema confirma a transação (commit).

Fluxo alternativo: se o saldo na origem for insuficiente, a operação é cancelada e nenhuma alteração é persistida.

8.3 UC08 — Ajustar estoque (fluxo detalhado)

Pré-condição: produto e localização cadastrados. O saldo atual pode ser zero (localização nunca teve esse produto).

- 1. Operador informa produto, localização e a quantidade contada fisicamente (e, opcionalmente, o motivo do ajuste).
- 2. Sistema calcula a diferença entre a quantidade contada e o saldo atual do sistema.
- 3. Se a diferença for zero, nenhuma movimentação é registrada.
- 4. Se a contagem for maior que o saldo, o sistema credita a diferença (mesma semântica de uma entrada).
- 5. Se a contagem for menor, o sistema debita a diferença (mesma semântica de uma saída) — nunca falha por saldo insuficiente, pois a diferença nunca excede o saldo atual.
- 6. Sistema registra uma MovimentacaoEstoque do tipo Ajuste, com o motivo informado.

Pós-condição: saldo do sistema passa a refletir exatamente a contagem física informada.

9. Regras de Negócio

Código	Regra
RN01	Uma movimentação de saída ou transferência não pode resultar em saldo negativo de estoque.
RN02	Toda movimentação de estoque é registrada em log auditável e imutável — não pode ser editada ou excluída.
RN03	O SKU de um produto é único em todo o sistema.
RN04	Uma localização pertence a exatamente um armazém; o código de uma localização só precisa ser único dentro do próprio armazém.
RN05	Um pedido de recebimento só é encerrado (Concluído) quando todos os itens forem totalmente recebidos.
RN06	Um pedido de expedição só pode ser expedido se houver saldo suficiente em estoque para todos os itens — tudo ou nada.
RN07	Apenas usuários com perfil Administrador podem gerenciar usuários e perfis.
RN08	Uma transferência gera débito na origem e crédito no destino dentro da mesma transação de banco de dados.
RN09	Um pedido (recebimento ou expedição) só pode ser cancelado enquanto estiver Pendente ou EmAndamento. Uma vez Concluído ou já Cancelado, a tentativa de cancelar é rejeitada.

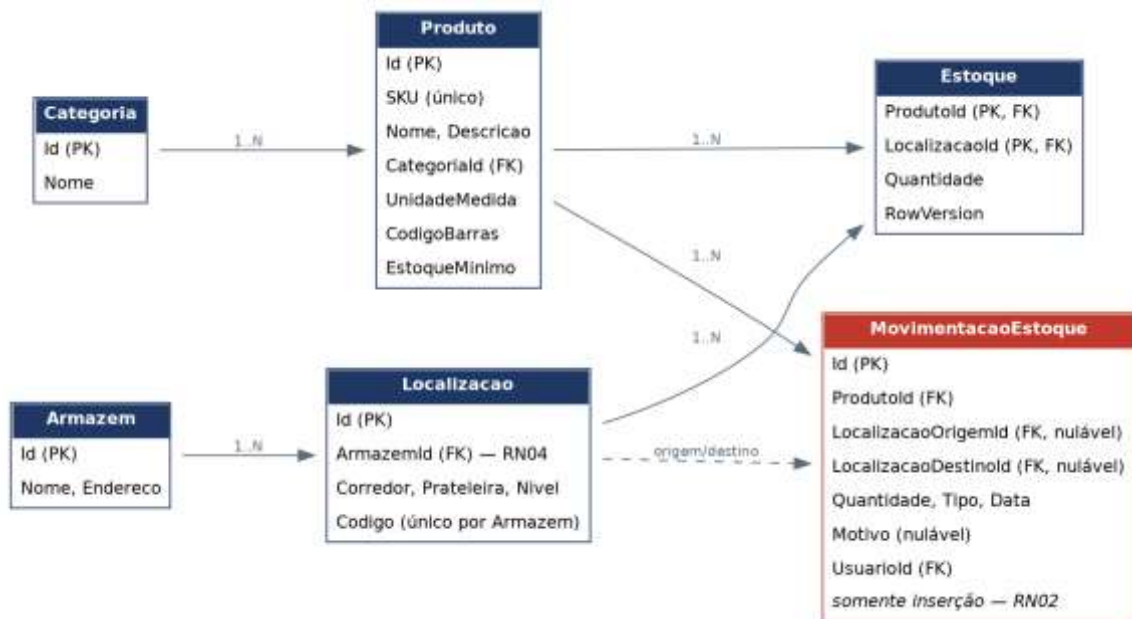
10. Modelo de Dados

Entidade	Campos principais
Produto	Id, SKU, Nome, Descricao, CategoriaId, UnidadeMedida,CodigoBarras, EstoqueMinimo
Categoria	Id, Nome
Armazem	Id, Nome, Endereco
Localizacao	Id, ArmazemId, Corredor, Prateleira, Nivel,Codigo
Estoque	ProdutoId, LocalizacaoId, Quantidade, RowVersion
MovimentacaoEstoque	Id, ProdutoId, LocalizacaoOrigemId, LocalizacaoDestinoId, Quantidade, Tipo, Data, Motivo, UsuarioId
Fornecedor	Id, Nome, CNPJ, Contato
PedidoRecebimento	Id, FornecedorId, Status, DataPrevista, DataRecebimento
ItemPedidoRecebimento	Id, PedidoRecebimentoId, ProdutoId, QuantidadeEsperada, QuantidadeRecebida
Cliente	Id, Nome, Documento, Contato
PedidoExpedicao	Id, ClienteId, Status, DataPrevista, DataExpedicao
ItemPedidoExpedicao	Id, PedidoExpedicaoId, ProdutoId, QuantidadeSolicitada, QuantidadeExpedida
Usuario	Id, Nome, Email, SenhaHash, PerfilId
Perfil	Id, Nome
RefreshToken	Id, UsuarioId, Token, ExpiraEm, Revogado, CriadoEm

Observações de modelagem:

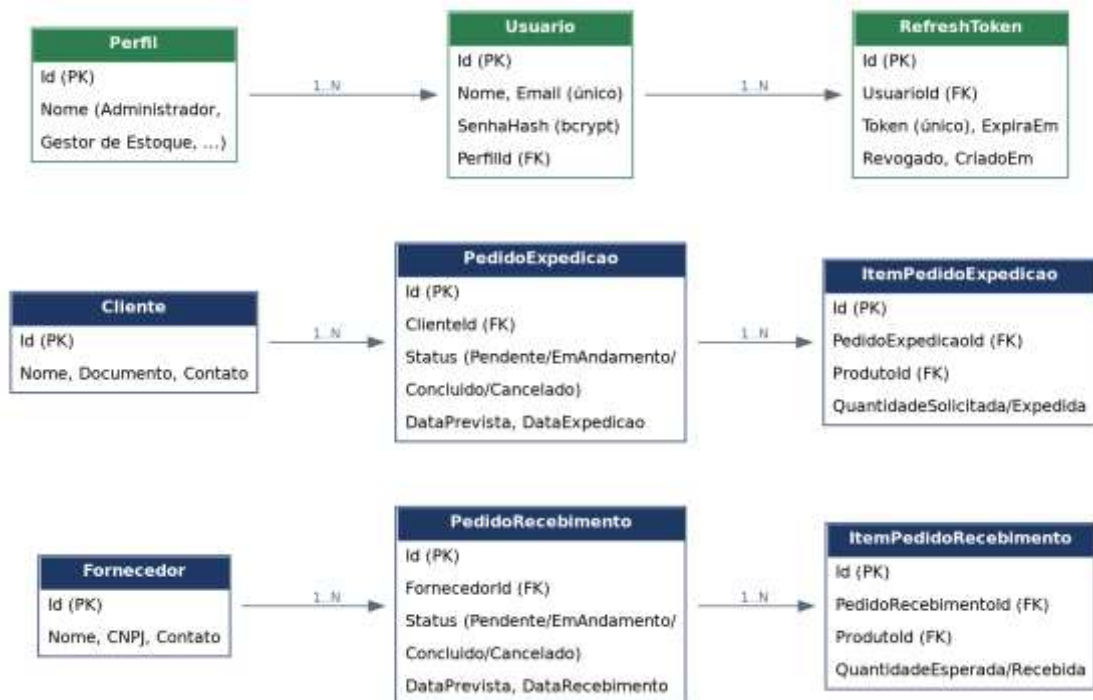
- Estoque possui chave composta (ProdutoId + LocalizacaoId) com índice único.
- MovimentacaoEstoque funciona como log de auditoria — somente inserção, nunca alteração; o campo Motivo é usado principalmente pelo Ajuste.
- RowVersion em Estoque permite controle de concorrência otimista em baixas simultâneas.
- Perfil não tem uma coluna de permissões granulares — a permissão é resolvida em código comparando o Nome do perfil (ex.: "Administrador") ao exigido pelo endpoint. Diferente do que a Versão 1.0 deste documento previa.
- RefreshToken é revogado (não excluído) ao ser usado (rotação) ou no logout — mantém histórico auditável de sessões.

10.1 Diagrama Entidade-Relacionamento — Núcleo de estoque



Categoria, Produto, Armazem, Localizacao, Estoque e MovimentacaoEstoque

10.2 Diagrama Entidade-Relacionamento — Pedidos e acesso



Fornecedor/Cliente, Pedidos e itens, Perfil, Usuario e RefreshToken

11. Contratos de API

Visão geral dos endpoints por recurso. Contratos completos (request/response, códigos de erro) estão documentados via Swagger/OpenAPI (rota /swagger em Development).

Recurso	Endpoints
Autenticação	POST /api/auth/login · POST /api/auth/refresh · POST /api/auth/logout · POST /api/auth/alterar-senha
Produtos	GET, GET /{id}, POST, PUT /{id}, DELETE /{id} — /api/produtos (paginado)
Categorias	GET, POST, PUT /{id}, DELETE /{id} — /api/categorias
Armazéns	GET, POST, PUT /{id}, DELETE /{id} — /api/armazens
Localizações	GET, POST, PUT /{id}, DELETE /{id} — /api/localizacoes
Estoque	GET /api/estoque?produtoId=
Movimentações	POST /entrada · /saida · /transferencia · /ajuste — /api/movimentacoes
Pedidos de Recebimento	GET, GET /{id}, POST, POST /{id}/itens/{itemId}/confirmar-recebimento, POST /{id}/cancelar — /api/pedidos-recebimento (paginado)
Pedidos de Expedição	GET, GET /{id}, POST, POST /{id}/expedir, POST /{id}/cancelar — /api/pedidos-expedicao (paginado)
Fornecedores / Clientes	GET, POST, PUT /{id}, DELETE /{id} — /api/fornecedores e /api/clientes
Relatórios	GET /api/relatorios/estoque-baixo · GET /api/relatorios/movimentacoes (com filtros)
Usuários / Perfis	GET, POST, PUT /{id}, DELETE /{id}, POST /{id}/resetar-senha — /api/usuarios (Administrador); GET, POST — /api/perfis (Administrador)
Observabilidade	GET /health (sem autenticação — testa conectividade com o banco)

12. Plano de Testes (critérios de aceite)

Critérios de aceite no formato Dado / Quando / Então para os fluxos mais críticos — todos cobertos por testes automatizados (unitários e/ou integração).

Cenário	Dado	Quando	Então
Entrada de produto	Produto e localização cadastrados	Operador registra entrada de 10 unidades	Saldo aumenta em 10 e uma movimentação Entrada é criada
Saída sem saldo suficiente	Saldo do produto na localização é 5	Operador tenta registrar saída de 8 unidades	Sistema rejeita a operação (422) e nenhum saldo é alterado (RN01)
Transferência entre localizações	Saldo de 20 unidades na origem	Operador transfere 10 unidades	Origem fica com 10, destino fica com 10, uma movimentação Transferência é criada (RN08)
Ajuste de estoque	Saldo do sistema é 20, contagem física encontrou 15	Operador registra ajuste com quantidade contada 15	Sistema debita 5, registra movimentação Ajuste com o motivo informado
Expedição sem saldo	Pedido de expedição com item sem saldo suficiente	Gestor tenta expedir o pedido	Sistema bloqueia a expedição inteira, nenhum item é baixado (RN06)

Cenário	Dado	Quando	Então
Cancelar pedido concluído	Pedido de recebimento já está Concluído	Gestor tenta cancelar o pedido	Sistema rejeita com erro claro (RN09); botão de cancelar não aparece na tela
Acesso restrito	Usuário sem perfil Administrador	Usuário tenta cadastrar um novo usuário	Sistema nega o acesso (403, RN07)
Renovação de sessão	Access token expirado, refresh token válido	Front faz uma chamada autenticada	Token é renovado automaticamente e a chamada original é refeita, sem exigir novo login

13. Riscos e Mitigações

Risco	Impacto	Mitigação
Concorrência em baixas simultâneas do mesmo item	Saldo incorreto de estoque	RowVersion (concorrência otimista) e transações no banco de dados — implementado.
Crescimento do log de movimentações	Degradação de performance em consultas históricas	Índices adequados; arquivamento de dados antigos permanece como melhoria futura.
Divergência entre schema do banco e entidades do código	Erros em tempo de execução (Dapper não valida em tempo de compilação)	Scripts de migração versionados (DbUp) e testes de integração contra banco real — implementado; foi assim que um bug de ordem de colunas foi pego.
Segredo JWT ou dados de seed vazando para produção	Qualquer pessoa poderia forjar tokens válidos, ou um usuário administrador com senha conhecida seria criado	API recusa subir fora de Development com a chave JWT padrão; CLI de migração não aplica o seed de dados a menos que --seed seja passado explicitamente — implementado.
Deploy em ambiente de produção	Sistema roda hoje apenas localmente	Fora do escopo desta entrega, por decisão do time do projeto — ver Seção 15.

14. Status do Projeto

Esta seção substitui o cronograma da Versão 1.0 (que era uma projeção antes do desenvolvimento começar) por um retrato do que foi efetivamente entregue.

Área	Status
Núcleo de estoque	Completo — produtos, categorias, armazéns, localizações, estoque, movimentações (entrada/saída/transferência/ajuste).
Pedidos	Completo — recebimento e expedição, incluindo confirmação item a item, expedição tudo-ou-nada e cancelamento.
Autenticação e permissões	Completo — login, refresh token com rotação, logout, alterar/resetar senha, perfis de acesso.
Relatórios	Completo — estoque baixo e histórico de movimentações com filtros.

Área	Status
Observabilidade e segurança	Completo — Serilog, health check, rate limiting no login, fail-fast de configuração insegura.
Testes	Completo — testes unitários e de integração no back-end; testes de página e componente no front-end.
Deploy em produção	Não iniciado — fora do escopo desta entrega.

15. Próximos Passos

Com o escopo funcional, de segurança e de testes concluído, o único item pendente é:

- Deploy em ambiente de produção (hospedagem, CI/CD, certificados, variáveis de ambiente/secrets reais) — decisão consciente de ficar fora desta entrega.

Melhorias possíveis para além do escopo atual, sem caráter de pendência:

- Arquivamento/particionamento do log de movimentações caso o volume cresça muito no futuro.
- Permissões mais granulares por perfil (hoje é binário: Administrador vs. demais).